

UNIDAD DIDÁCTICA 5

DESARROLLO DE COMPONENTES PARA UN SISTEMA EPR-CRM

**MÓDULO PROFESIONAL:
SISTEMAS DE GESTIÓN EMPRESARIAL**



CESUR
Tu Centro Oficial de FP

CONTENIDO

RESUMEN INTRODUCTORIO	2
INTRODUCCIÓN	2
CASO INTRODUCTORIO	3
1. ARQUITECTURA DEL ERP-CRM	4
1.1 Técnicas y estándares.....	5
1.2 Especificaciones funcionales para el desarrollo de componentes	6
2. DESARROLLO DE MÓDULOS	9
2.1 Entornos y herramientas de desarrollo en sistemas ERP y CRM.....	10
2.2 Lenguaje proporcionado por los sistemas erp-crm	14
2.2.1 Características y sintaxis del lenguaje.....	14
2.2.2 Declaración de datos.....	15
2.2.3 Operadores.....	16
2.2.4 Estructuras de programación y sentencias del lenguaje	17
2.3 Formularios e informes en sistemas ERP-CRM.....	18
2.3.1 Llamadas a funciones, librerías de funciones (APIs)	19
2.3.2 Elementos principales	20
2.3.3 Inserción, modificación y eliminación de datos en los objetos	23
2.3.4 Acciones y menús.....	25
2.3.5 Seguridad.....	26
2.4 Procesamiento de datos y obtención de la información. Procesamiento de datos	29
2.4.1 Herramientas para la creación de formularios e informes	29
2.4.2 Diseño de informes	30
2.5 Operaciones de consulta. Herramientas	34
2.5.1 Técnicas de optimización de consultas y acceso a grandes volúmenes de información	37
2.5.2 Generación de programas de extracción de datos entre sistemas (batch inputs)	38
2.6 Depuración y tratamiento de errores.....	40
2.6.1 Manejo de errores.....	42
RESUMEN FINAL	45

RESUMEN INTRODUCTORIO

En esta unidad, finalizamos el estudio en la implantación de un ERP-CRM profundizando en los conceptos básicos y herramientas sobre desarrollos de componentes y módulos. Comenzaremos dando las pautas sobre cuando una empresa se plantea la realización de desarrollos a medida y cómo puede planificarlas.

En segundo lugar, entraremos de lleno en la realización de un nuevo módulo dentro de Odoo, cuáles son las herramientas necesarias y los ficheros implicados. Un módulo incorpora modelos que y controladores que podrán ser después utilizados en diferentes vistas como formularios, informes o gráficos.

Por último, introducimos los conceptos sobre depuración y cuáles son las metodologías y herramientas que disponemos para la gestión de los errores.

INTRODUCCIÓN

Dentro de un entorno en constante evolución, donde se encuentran inmerso el sector empresarial, la tecnología juega un papel fundamental para la adaptación a cambios y mejora de la competitividad.

Los sistemas ERP-CRM no solo permiten integrar la información y automatizar procesos, sino que también ofrecen la posibilidad de desarrollar componentes y módulos específicos que amplíen sus funcionalidades, por lo que, su utilización proporciona una solución flexible, capaz de ajustarse a su realidad y crecer al mismo ritmo que ella.

El uso de estos sistemas no se limita a grandes corporaciones, están presentes en sectores muy diversos, desde la industria y la logística, hasta el comercio minorista, la hostelería, la sanidad o la educación.

Gracias a su versatilidad, pueden adaptarse tanto a pequeñas empresas como a grandes multinacionales, aportando valor en cualquier ámbito donde sea necesario coordinar procesos y disponer de información clara y accesible.

Su futuro, pasa por caminar en paralelo con la inteligencia artificial, el análisis avanzado de datos y el trabajo en la nube, lo que permitirá a los ERP-CRM ayudar a saber qué está pasando en la empresa, además de también, anticiparse a lo que puede ocurrir y proponer soluciones de manera automática, constituyendo estos sistemas en aliados estratégicos que acompañan a las organizaciones en su crecimiento y en su capacidad de innovar.

CASO INTRODUCTORIO

Como desarrollador de aplicaciones perteneces al departamento de informática de una cadena de franquicias dentro del sector de restauración y hostelería con 15 restaurantes temáticos a lo largo del país.

Se ha comenzado realizando una instalación del sistema ERP de Odoo en su versión community en un servidor para poder realizar pruebas, se han configurado varios usuarios, entre ellos uno de administrador y desarrollador.

Junto a tu jefe de departamento necesitáis realizar una evaluación de un desarrollo a medida, ya que el proceso de procesado de productos en cocina y distribución a las diferentes franquicias no viene como un estándar.

Al final de esta unidad tendremos los conocimientos para poder realizar un nuevo módulo a medida, así como sus vistas pertinentes.

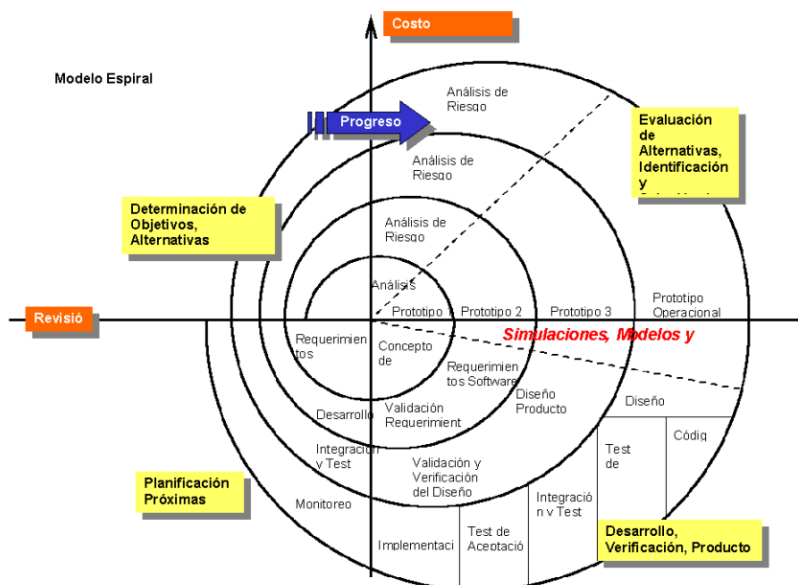
1. ARQUITECTURA DEL ERP-CRM

Tras detectar que el proceso de cocina y distribución no está cubierto por el ERP estándar, deberás analizar si conviene adaptar Odoo con un módulo a medida o mediante configuraciones híbridas.

Realizarás una comparación de estas opciones, para valorar pros y contras en relación a la cadena de restaurantes y harás la propuesta de metodología de trabajo más adecuada para llevar a cabo el proyecto.

Tal y como hemos desarrollado e introducido a lo largo del curso en anteriores unidades didácticas, las empresas tienen necesidades diversas que hacen que la implantación de un sistema de gestión empresarial ERP-CRM necesite en la gran mayoría de ocasiones de adaptaciones para poder dar respuesta a los diferentes procesos y operaciones que se realizan dentro del negocio.

Además, estas necesidades son dinámicas, de manera que van evolucionando junto con la propia empresa. En un primer análisis se recogen los requisitos iniciales, pero durante las pruebas, la implantación e incluso en el uso real del sistema pueden aparecer nuevos requerimientos y funcionalidades que será necesario incorporar.



Modelo en espiral desarrollado por Boehm.

Fuente:

https://blogger.googleusercontent.com/img/b/R29vZ2xl/AVvXsEj9voLCzyo2s8e3881ij5KyBOJl_zwwsDQtY5uHammxNMEzF3SunqgfZKDglsfQTM9niw-82EbGvFu4SSR_lbvJQCvuzB9r65b1hna3Ik9NC5tMpCHu7C1Kn2IfGm1RLxajawmoTs_uBII/s1600/iaaaarasrsat.png

Como vemos en la anterior imagen, un posible modelo de planificación de proyectos consiste en un modelo en espiral donde las fases se plantean una y otra vez al pasar por el mismo estado, pero en diferentes momentos de la vida de una empresa.

1.1 Técnicas y estándares

Podemos entender el software empresarial desde tres puntos de vista:

- El software standard es el que se ha pensado y desarrollado para poderse implantar en cualquier tipo de empresa, sector o actividad, intentando que se cubran los procesos de gestión más habituales dentro de las empresas. Dentro de estos softwares generalistas nos encontramos las llamadas verticalizaciones que consiste en una especialización dentro de un determinado sector. Por ejemplo, una verticalización de software de gestión dentro de las empresas del sector de la moda puede llevar desarrollado de serie muchos aspectos sobre el tallaje o la venta.
- El siguiente paso dentro de los diferentes tipos de software de gestión tenemos el software híbrido. Durante la captura de necesidades dentro de una empresa se recogen esas necesidades que se convierten en adaptaciones a partir de los diferentes módulos que tiene el software, donde esas adaptaciones a su vez pueden ser de dos niveles, modificaciones sobre ventanas, vistas e informes existentes, o generación de nuevos elementos que se puedan añadir a nuestro sistema.
- Por último, el software a medida es la tercera opción que nos encontramos de tipología de software de gestión. En este caso el planteamiento desde el principio es el desarrollo del software desde cero y a medida para los procesos de negocio de la empresa en concreto. Este caso, debe ser tomado como recurso después de haber realizado un análisis minucioso de las opciones que ofrece el mercado, ya que el desarrollo a medida normalmente es costoso en tiempo y en recursos.

Las técnicas y estándares que se utilizan en la adaptación y desarrollo a medida de software de gestión empresarial son las mismas que en el desarrollo de software.

Se utilizan lenguajes de programación libre o código abierto, como es el caso de Odoo que utiliza los siguientes lenguajes de programación:

- Lenguaje Python para el desarrollo de la lógica computacional.

- Lenguajes HTML, JS y CSS para el desarrollo visual y de interacción con el usuario.
- PostgreSQL para la gestión de la base de datos.

Una de las metodologías que más de moda nos encontramos actualmente y más utilizadas para la planificación del desarrollo software son las metodologías ágiles, como por ejemplo: SCRUM, Kanban, Extreme Programming, Lean Software Development, que son el estándar en el desarrollo de software de gestión, frente a los modelos clásicos en cascada, al priorizar la entrega continua, flexibilidad y feedback del cliente.



ENLACE DE INTERÉS

Es esencial visitar el manifiesto ágil para comenzar a comprender las metodologías de gestión de proyectos como SCRUM:



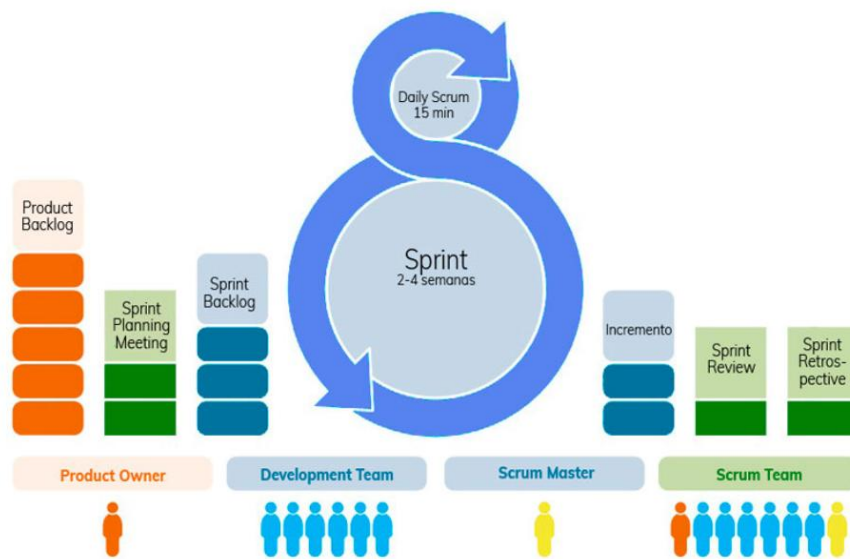
1.2 Especificaciones funcionales para el desarrollo de componentes

El esfuerzo que supone realizar un documento de especificaciones detallado junto con los requisitos es muy grande. Es importante tener dentro de los hitos este objetivo, pero puede que ni el cliente ni nosotros si somos los generadores del proyecto, sepamos de todas las necesidades a priori.

En muchas ocasiones, los usuarios necesitan ver físicamente el producto para poder plantear necesidades o modificaciones, y estas necesidades puede que en muchos casos aparezcan durante el uso del software, y más en el caso del software de gestión empresarial que tiene tantos pequeños detalles que repercuten enormemente en el funcionamiento final.

Aquí cobran relevancia las metodologías ágiles, especialmente SCRUM, con su Product Backlog (lista viva de requisitos) y los Sprints (ciclos cortos de desarrollo), de modo que,

permiten un enfoque iterativo para validar partes del sistema en pocas semanas, recibiendo retroalimentación constante del cliente y ajustando prioridades.



Metodología SCRUM.

Fuente: <https://jorgesaiz.com/blog/proyecto-scrum-una-explicacion-sencilla/>

Como vemos en la imagen y de una forma sencilla, en SCRUM los requisitos se resuelven con el elemento denominado Product BackLog. Es una lista viva de los requisitos funcionales y no funcionales que dan valor a un cliente y al producto y que además se encuentran priorizados.

Sin entrar en el detalle del funcionamiento de SCRUM, esta lista es viva ya que a lo largo del desarrollo del proyecto van actualizándose o modificándose a lo largo de lo que se denominan Sprints.

Un Sprint como vemos tiene la capacidad de planificar parte del proyecto a lo largo de un período de semanas y que nos permite hacer visible parte del proyecto.

Esa visibilidad a su vez nos permite un feedback sobre parte del proyecto desde el punto de vista del cliente y los usuarios y por lo tanto poder modificar esos requisitos iniciales y las planificaciones futuras en forma de nuevos sprints.



EJEMPLO PRÁCTICO

En la empresa donde estas trabajando como desarrollador de software, vas a realizar un nuevo módulo en Odoo para gestionar Menús y Platos. Para ello utilizarás la metodología SCRUM y necesitas obtener los requisitos del software para construir nuestro producto backlog.

Los pasos a seguir son:

1. Reunión:

En el equipo de elaboración de los requisitos se deben reunir, tanto el cliente (Product Owner) como el responsable de desarrollo (Scrum Master). En nuestro este caso, el Product Owner será el director de restaurantes y franquicias, y como Scrum Master el jefe de departamento técnico.

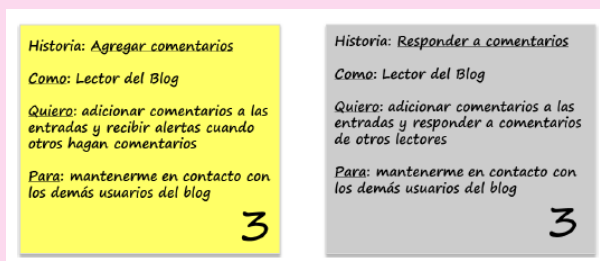
El objetivo de la reunión es, que el cliente explique qué necesita y el equipo técnico traduzca esas necesidades a tareas concretas.

2. Historias de usuario:

En segundo lugar, recogeremos todas las tareas (historias de usuario en argot SCRUM), con un título suficientemente claro y específico, y un detalle del mismo, para que los requisitos técnicos se expresen desde el punto de vista del usuario final.

Un ejemplo de un requisito tendría las siguientes partes:

- Historia: Crear un plato.
- Como: Cocinero.
- Quiero: Introducir todos los ingredientes de un plato con un nombre y título de plato.



Ejemplo de historia de usuario.

Fuente: http://www.blmovil.com/wp-content/uploads/sites/3/2019/04/Scrum_HU6.png

3. Priorización del backlog:

Todas las historias de usuario se guardan en el Product Backlog (lista de tareas pendientes) Priorizamos la lista de requisitos (historias de usuario), bien de una forma secuencial, bien agrupándola o bien mediante una estimación de tiempos, con criterios como: Importancia de negocio, urgencia de uso o esfuerzo estimado de desarrollo. Permitiendo, de este modo, decidir qué historias se implementarán primero en los siguientes Sprints.

Área de requisitos	Requisitos	Origen	Valor	Estimación inicial	Factor Ajuste	Estimación ajustada	Iteración:	1	2	3	4	5
Área X	Requisito A	Marketing	2000	15		15	Pendiente:	225	170	114	57	
Área Z	Requisito B	Producción	1750	20		20						
Área Y	Requisito C	Ventas	1500	20		20						
	Iteración 1		5250	55		55		55	0	0	0	0
Área Z	Requisito C	Producción	1250	15	0,2	18		18	18	0		
Área X	Requisito D	Producción	1250	20		20		20	20	0		
Área Z	Requisito E	Marketing	1000	15	0,2	18		18	18	0		
	Iteración 2		3500	50		58		58	58	0	0	0
	Primera entrega		8750	105		111		111	56	0	0	0
Área X	Requisito F	Marketing	1250	20	0,2	24		24	24	24	0	
Área Y	Requisito G	Marketing	750	15		15		15	15	15	0	
Área Y	Requisito H	Ventas	750	15	0,2	18		18	18	18	0	
	Iteración 3		2750	50		57		57	57	57	0	0
Área Z	Requisito I	Producción	700	15	0,2	18		18	18	18	18	
Área Y	Requisito J	Marketing	500	10	0,5	15		15	15	15	15	
Área Y	Requisito K	Ventas	500	20	0,2	24		24	24	24	24	
	Iteración 4		1700	45		57		57	57	57	57	0
	Segunda entrega		4450	95		114		114	114	114	57	0

Ejemplo de priorización.

Fuente: <https://proyectosagiles.org/wp-content/uploads/2015/09/proyectos-agiles-product-backlog-alfa-200912011.gif>



VÍDEO DE INTERÉS

A continuación, se muestra un vídeo muy explicativo de las partes más importantes de la metodología SCRUM:



2. DESARROLLO DE MÓDULOS

Vas a ser el encargado de preparar un prototipo de módulo en Odoo que permita controlar el procesamiento de productos en cocina y su envío a las distintas franquicias.

Deberás diseñar qué información debe gestionarse (ingredientes, tiempos, stock, destinos), qué vistas serán necesarias para los usuarios y definir cómo comprobarás que el módulo funciona correctamente en las pruebas iniciales.

Una solución híbrida tal y como acabamos de estudiar es una opción fantástica y muy válida para poder tener lo mejor de las soluciones a medida y de las soluciones estándar.

Cuando una empresa o un negocio se plantea implantar un ERP-CRM realiza un proceso de estado del arte de este tipo de software, al mismo tiempo que realiza un estudio de

las necesidades que transforma en unas especificaciones para el proyecto de implantación.

Aun cuando este proceso se realice muy minuciosamente y realizando pertinentes pruebas sobre el software, se producen durante la implantación y sobre todo durante el uso del software de gestión momentos en los que se observa necesidades nuevas no tenidas en cuenta. Estas nuevas necesidades que no proporciona el software se pueden resolver de dos formas con Odoo, mediante modificaciones de los formularios, informes o vistas tal y como hemos visto anteriormente, o mediante el desarrollo de nuevos módulos.

Un módulo dentro de Odoo nos permite modificar y personalizar el comportamiento de la instalación estándar de Odoo, añadiendo nuevas funcionalidades o alterando funcionalidades existentes que ya teníamos con otros módulos.

En la actualidad, disponemos de Odoo v16, donde estas personalizaciones suelen hacerse mediante nuevas vistas XML/OWL, informes QWeb y desarrollos en Python.



RECUERDA

En la UD2 introdujimos el proceso de implantación y dentro de este las medidas a tener en cuenta, medidas como la parametrización o la adaptación.

En la UD4 vimos cómo se realizaba el proceso de parametrización y adaptación dentro de Odoo, tanto con las tablas y vistas añadiendo nuevos elementos como con los informes modificándolos.

2.1 Entornos y herramientas de desarrollo en sistemas ERP y CRM

Para el desarrollo de un nuevo modelo dentro de Odoo necesitaremos las siguientes herramientas y entornos, algunos de ellos ya los hemos trabajado en unidades anteriores:

- Odoo community, el framework nos proporciona lo esencial para poder comenzar a realizar las pruebas y desarrollos ya que tal y como hemos visto es un software de gestión empresarial de código abierto.

- Un IDE de desarrollo que nos permita editar el código, así como nos proporcione utilidades para realizar esa tarea de forma cómoda. En nuestro caso usaremos Visual Code con diversos widgets/plugins instalados para realizar dichos desarrollos. Hemos de tener en cuenta que para realizar las modificaciones y crear nuevos módulos nos encontramos con múltiples lenguajes, por lo que Visual Code será una herramienta perfecta para llevar a cabo esos desarrollos.
- Compiladores y entornos de desarrollo relacionados con los lenguajes a programar. Algunos lenguajes, en nuestro caso Python, necesitan de instalaciones añadidas para poder realizar las pruebas, depuraciones y compilaciones del código desarrollado. Además de PostgreSQL y librerías incluidas en el fichero `requirements.txt` de Odoo.

Tal y como comentamos, Visual Studio Code es un entorno de programación modular, ligero y gratuito que nos permite realizar programación con diferentes lenguajes de programación.

En la actualidad, para realizar pruebas y simulaciones, es recomendado el uso de Docker y Docker Compose para desplegar Odoo y PostgreSQL en contenedores, al facilitar la creación, prueba y borrado de entornos de desarrollo de manera rápida y segura.

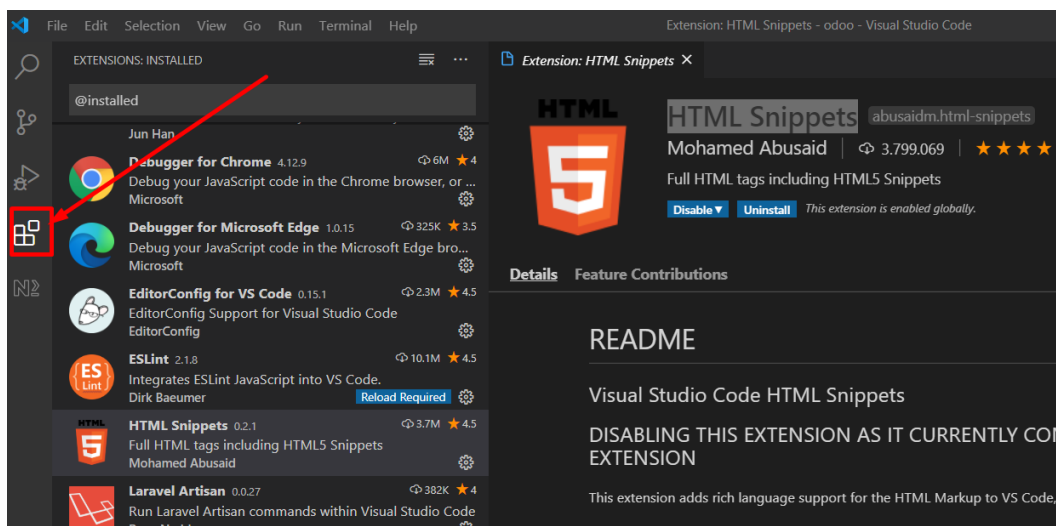


PARA SABER MÁS

Enlace a la página del fabricante de Visual Studio Code y su descarga:



En nuestro caso, dentro del desarrollo de módulos de Odoo necesitaremos instalar diferentes plugins que nos faciliten las tareas como desarrolladores. Para ello una vez abierto Visual Code nos dirigiremos a la zona de gestión de extensiones tal y como vemos en la imagen:



Extensiones Visual Code.

Fuente: imagen propia

Dentro de las extensiones recomendadas que debemos instalar son:

- Extensiones específicas: Python (ms-python.python) y Pylance.



ENLACE DE INTERÉS

Desde el propio entorno de desarrollo, Visual Code Studio podemos realizar la instalación de los diferentes plugins de desarrollo, Python:



- Extensiones para XML y HTML: XML Tools, HTML Snippets.



ENLACE DE INTERÉS

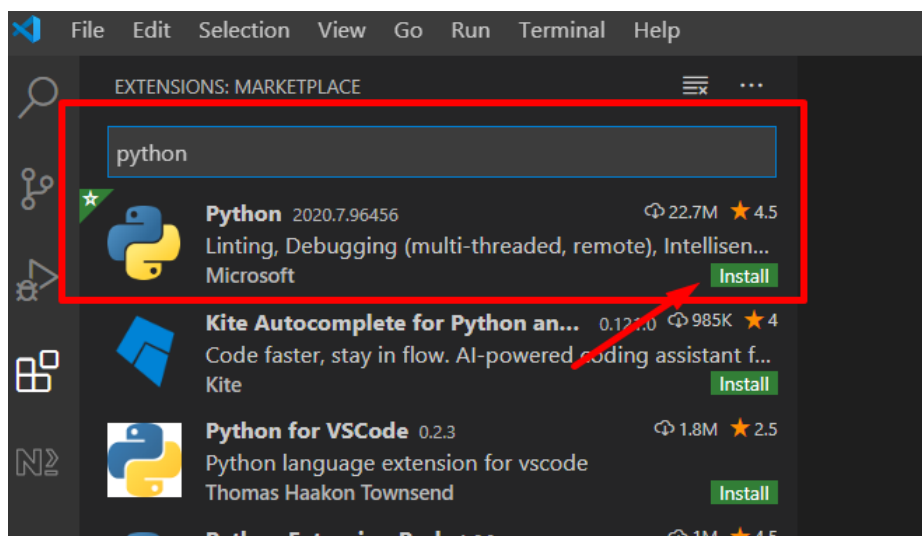
Desde el propio entorno de desarrollo, Visual Code Studio podemos realizar la instalación de los diferentes plugins de desarrollo, HTML:



- Extensión para JavaScript: ESLint.
- Extensiones adicionales recomendadas: EditorConfig y Odoo Snippets (comunidad).

Un ejemplo de instalación de los plugins lo podemos tener con el plugin de Python:

1. Haremos clic sobre el icono de extensiones de Visual Code Studio.
2. En la búsqueda de extensiones escribiremos Python e instalaremos la extensión denominada Python.



Instalación de Python.

Fuente: imagen propia.

3. Es recomendable cambiar la versión de Python que estamos usando en nuestro configurador de extensión. Teclear (CTRL+Shift+P), aparecerá un menú sobre el

que escribiremos, Python: Select Interpreter. Seleccionaremos el intérprete de Python instalado.



VÍDEO DE INTERÉS

En el siguiente vídeo tenemos cómo utilizar Visual Studio con Python:



2.2 Lenguaje proporcionado por los sistemas ERP-CRM

Son muchos los lenguajes de programación que existen y utilizan los diferentes software de gestión empresarial. En el caso de Odoo los diferentes lenguajes de programación que se usan son:

- HTML y CSS, son lenguajes de marcas usados en el desarrollo de aplicaciones web. En el caso de Odoo estos 3 lenguajes se utilizan para el desarrollo de la parte frontal o de vistas de la aplicación. No son lenguajes de programación estrictamente. se emplean junto a XML para definir formularios, menús e informes, y en versiones recientes también se utiliza OWL (Odoo Web Library), un framework basado en JavaScript para construir vistas dinámicas.
- JavaScript, es un lenguaje de programación interpretado por el navegador que también nos sirve para definir la parte frontal y de vistas de nuestra aplicación.
- Python, es un lenguaje de programación de alto nivel, interpretado y orientado a objetos. En el caso de Odoo, se utiliza el lenguaje para todos los procesos de lógica.

2.2.1 Características y sintaxis del lenguaje

Entre sus principales características destacan:

- **Portabilidad:** es posible ejecutarlo en diversas plataformas como Linux, Windows o macOS.
- **Facilidad de uso:** posee una sintaxis muy sencilla por lo que es muy fácil su aprendizaje.
- **Flexibilidad:** dispone de interfaces a llamadas al sistema y bibliotecas.
- **Interactividad.**
- **Gratuidad** y libre distribución.
- **Orientación a objetos:** como hemos comentado es un lenguaje orientado a objetos y por tanto dispone de herencia múltiple, polimorfismo, ligadura dinámica, así como todas las características de los lenguajes OO.
- **Lenguaje de alto nivel:** y como tal, facilita el no preocuparnos de detalles de bajo nivel tales como manejar la memoria empleada por el programa.
- **Ampliable:** dispone de multitud de librerías, tipos de datos y funciones incorporadas que facilitan la realización de tareas habituales sin necesidad de programarlas desde el inicio.



PARA SABER MÁS

En el siguiente enlace encontramos un manual de Python para principiantes que hace un recorrido por toda la sintaxis y usos de este lenguaje:



2.2.2 Declaración de datos

Los tipos de datos en Python se dividen en mutables, si su contenido se puede modificar, e inmutables, si su contenido no puede alterarse.

- **Números:** todos son tipos de datos inmutables:
 - Enteros (int).
 - Reales (float).
 - Complejos (complex).
- **Constantes lógicas** (bool): todas son del tipo inmutables.
- **Secuencias.**
 - Cadenas (string): pueden estar delimitadas por comillas simples o dobles ('Hola', "Hola").
 - Tuplas (tuple): se delimitan por paréntesis y sus elementos se indican separados por comas (a,b,c).
 - Listas (list): se delimitan por corchetes y sus elementos se indican separados por comas [1, 2, 3].
 - Las cadenas y tuplas son datos del tipo inmutables y las listas mutables.
- **Conjuntos:** colecciones no ordenadas de elementos (inmutables) que no se pueden encontrar duplicados.
 - Para que el conjunto sea mutable utilizamos la sentencia: set(secuencia) la cual crea un conjunto con los elementos de la secuencia especificada descartando las duplicaciones.
 - Para que el conjunto sea inmutable utilizamos la sentencia: frozenset(secuencia) la cual crea un conjunto con los elementos de la secuencia especificada descartando las duplicaciones.
- **Otros tipos de datos:** la constante None se utiliza para especificar un dato nulo.

2.2.3 Operadores

Sin entrar en profundizar en el uso de Python, las operaciones y estructuras dentro de Python son:

- **Comparadores lógicos:** en la siguiente imagen nos encontramos con un resumen de los diferentes comparadores.

<i>Operador</i>	<i>Significado</i>
<	Menor estricto que
<=	Menor o igual que
>	Mayor estricto que
>=	Mayor o igual que
==	Igual que
!=	Distinto que
is	Idéntico a
is not	No idéntico a

Comparadores en Pytho.

Fuente: Coremsa.

- **Operadores lógicos:** en Python existen tres operadores lógicos o booleanos: not, and, or.

<i>Declaración</i>	<i>Evalúa a</i>
bool(expr)	True si expr es verdadera, False en caso contrario
not expr	True si expr es falsa, False en caso contrario
expr1 or expr2	False si expr1 y expr2 son falsos, True en caso contrario
expr1 and expr2	True si expr1 y expr2 son verdaderos, False en caso contrario

Operadores lógicos en Python.

Fuente: Coremsa.

- **Operadores de asignación:** permiten la asignación de elementos.

2.2.4 Estructuras de programación y sentencias del lenguaje

Dentro de las posibilidades que nos encontramos dentro Python vamos a tratar de forma resumida tres de los aspectos más importantes.

- **Control de flujo:** son estructuras de programación que nos encontramos en todos los lenguajes y que nos permiten definir y controlar el flujo de la ejecución de las sentencias. En la siguiente imagen tenemos un resumen de las estructuras más importantes dentro de Python.

Declaración	Significado
if condicion: consecuencias elif condicion: consecuencias else : alternativas	Condicional simple (if), doble (if/else) y múltiple (if/elif/else)
while condicion: acciones	Bucle mientras
for elt in secuencia: acciones	Bucles para y desde
break continue	Interrupción de un bucle Continuación de un bucle

Estructuras de control de flujo dentro de Python.

Fuente: Coremsa.

- **Definición de funciones**, para definir una función en Python utilizamos la siguiente sintaxis:

```
def nombre_funcion(parametros):
    #comentarios
    Acciones
    return resultado
```

- Parámetros: sucesión de identificadores separados por comas que se le pasan a la función para que realice su menester.
- Return: devuelve el resultado de realizar la función con los parámetros pasados. Si no se especifica, la función devuelve None y en este caso se denomina procedimiento.

```
def restaComprobada (a=None, b=None):
    if a == None or b == None:
        print "Error, debes enviar dos números a la función"
        return None
    else:
        return a - b
```

2.3 Formularios e informes en sistemas ERP-CRM

La generación de nuevos formularios e informes pasa por conocer cómo funcionan los módulos dentro de Odoo, y para ello la construcción de un nuevo módulo en Odoo es una buena práctica para conocer todos los aspectos. Tanto el servidor como las extensiones de cliente en Odoo se empaquetan en módulos que a su vez se carga a través de la base de datos.

Las principales partes de un módulo en Odoo son:

- **Objetos de negocio o lógica de negocio.** Se declaran como clases de Python y son recursos que se hacen persistentes (se cargan) a través de configuración.
- **Vistas.** Donde se definen los interfaces con el usuario.
- **Ficheros de configuración y datos.** Son ficheros XML o CSV que definen metadatos sobre las vistas, los informes, la configuración y otros. Los hemos utilizado en unidades anteriores para modificar vistas, por ejemplo.
- **Controladores web.** Declaran las rutas que el navegador puede realizar.
- **Datos estáticos** como imágenes, css o javascript usados para diferentes puntos del cliente/servidor.

2.3.1 Llamadas a funciones, librerías de funciones (APIs)

Como comentábamos al indicar las características de Python, una de las principales ventajas de este lenguaje es la multitud de funciones que tienes disponibles en forma de bibliotecas, o como se llaman en Python “módulos”. Si queremos acceder a una función que se encuentra en una de las bibliotecas de Python debemos seguir los siguientes pasos:

1. Importar la biblioteca:

`import nombre_biblioteca` Ejemplo: `import math`

2. Acceder a la función cuando la necesitemos tan solo escribiendo el nombre de esta precedido por el nombre de la biblioteca en la que se encuentra y separados por un punto.

`Nombre_biblioteca.nombre_funcion(parametros)` Ejemplo: `print math.sin(2.0)`

Una alternativa al método expuesto anteriormente sería el importar de esta cada vez que queramos utilizar una de sus funciones.

`From nombre_biblioteca import*` Ejemplo: `from math import *`

También es posible importar varias librerías a la vez en una sola sentencia tan solo separándolas por comas.

Tal y como veremos durante la creación de los módulos, odoo utiliza una serie de paquetes que deben importarse en el entorno para su correcto funcionamiento. Usaremos la herramienta de Python pip para realizar esta tarea:

```
pip install Jinja2
```



2.3.2 Elementos principales

Cada módulo es un directorio dentro del directorio principal de módulos.

Un módulo tiene dos ficheros principales de definición:

- El manifest
- El fichero de importaciones o paquetes de Python definido dentro del fichero `__init__.py`

Para la creación de un nuevo módulo usaremos el siguiente comando:

```
odoo-bin scaffold <nombre del módulo> <subdirectorio>
```

Para nuestro ejemplo, en el caso de Windows 10, y desde el directorio C:\Program Files (x86)\Odoo 13.0\server, ejecutaremos:

```
./odoo-bin scaffold mimodulo addons
```

En el caso de utilizar Odoo 16, desde el directorio raíz de instalación o dentro de un contenedor Docker, ejecutaremos:

```
odoo-bin scaffold mimodulo /mnt/extra-addons
```

Puede que durante el uso de odoo-bin, aparezca el siguiente error:

```
pacog@DESKTOP-FMSL9H7 MINGW64 /c/Program Files (x86)/Odoo 13.0
$ ./server/odoo-bin scaffold menu addons
Traceback (most recent call last):
  File "./server/odoo-bin", line 5, in <module>
    import odoo
  File "C:\Program Files (x86)\Odoo 13.0\server\odoo\__init__.py", line 132, in <module>
    from . import cli
  File "C:\Program Files (x86)\Odoo 13.0\server\odoo\cli\__init__.py", line 11, in <module>
    from . import scaffold
  File "C:\Program Files (x86)\Odoo 13.0\server\odoo\cli\scaffold.py", line 9, in <module>
    import jinja2
ModuleNotFoundError: No module named 'jinja2'

pacog@DESKTOP-FMSL9H7 MINGW64 /c/Program Files (x86)/Odoo 13.0
$
```

Error por falta de paquetes.

Fuente: elaboración propia.

Tal y como hemos visto en el apartado anterior, deberíamos usar pip para la instalación de los diferentes paquetes necesarios para usar odoo-bin:

Instalar todos los paquetes requeridos a partir del fichero de requirements: `pip install -r ./server/requirements.txt`

Instalar un paquete: `pip install Jinja2`



EJEMPLO PRÁCTICO

Como desarrollador de software dentro de una cadena de restaurantes eres el encargado del mantenimiento y evolución del software de gestión empresarial instalado en nuestra empresa, en este caso Odoo v13.

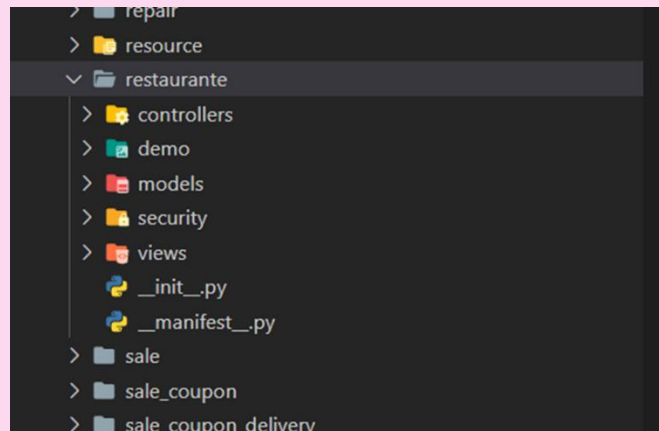
Debes realizar unas pruebas para añadir un módulo de platos de menú dentro de la aplicación que permita realizar justamente listados de productos que en este caso son los platos.

Para ello deberás seguir los siguientes pasos:

1. Crear la estructura del módulo

Odoo nos ofrece un comando para generar la plantilla de un módulo automáticamente. Usaremos: `./server/odoo-bin scaffold restaurante "C:\Program Files (x86)\Odoo 13.0\server\odoo\addons` para crear el nuevo módulo.

El comando `scaffold` crea una carpeta con todos los ficheros básicos que necesita un módulo (manifest, modelos, vistas, seguridad, datos de ejemplo...). Es como recibir un “esqueleto” que luego iremos rellenando.



Módulo creado menús.

Fuente: elaboración propia.

2. Editar el archivo de configuración (`\_\_manifest\_\_.py`)

El archivo (`\_\_manifest\_\_.py`) describe el módulo y le dice a Odoo cómo cargarlo.

Adaptamos el fichero (`\_\_manifest\_\_.py`), modificando los apartados:

- summary
- description
- author
- website
- category

```
# -*- coding: utf-8 -*-
{
    'name': "restaurante",

    'summary': """Modulo de platos y menus""",

    'description': """
        Módulo para manejar:
        - platos
        - menus
        """,

    'author': "Paco Gomez",
    'website': "http://www.repositoriocompartido.com",

    # Categories can be used to filter modules in modules listing
    # Check https://github.com/odoo/odoo/blob/13.0/odoo/addons/base/data/ir_module_category_data.xml
    # for the full list
    'category': 'Test',
    'version': '0.1',
}
```

```
# any module necessary for this one to work correctly
'depends': ['base'],

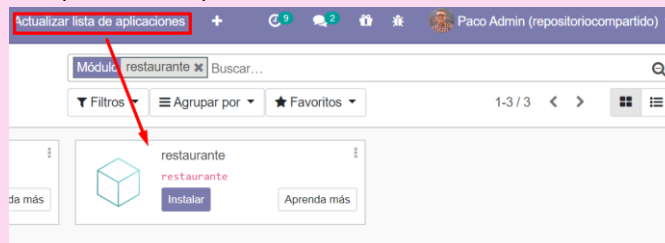
# always loaded
'data': [
    # 'security/ir.model.access.csv',
    'views/views.xml',
    'views/templates.xml',
],
# only loaded in demonstration mode
'demo': [
    'demo/demo.xml',
],
}
```

De tal manera que en el código los siguientes elementos proporcionan:

- `summary` y `description` --> resumen y explicación que se muestran en Odoo.
- `author` y `website` --> información del desarrollador o empresa.
- `category` --> clasifica el módulo para encontrarlo fácilmente.
- `depends` --> indica otros módulos que son necesarios para que este funcione.
- `data` y `demo` --> listas de archivos XML que definen vistas, menús y datos de prueba.

3. Actualizar la lista de aplicaciones de Odoo

Para poder visualizar e instalar en el futuro el módulo como desarrolladores deberemos actualizar el listado de módulos en la pantalla de aplicaciones.



Actualizar lista de aplicaciones.

Fuente: elaboración propia.

Con este último paso se indica a Odoo “búscame nuevos módulos en las carpetas de addons”, de modo que, si no actualizamos, el sistema no detectará el nuevo módulo creado.

2.3.3 Inserción, modificación y eliminación de datos en los objetos

Mediante la capa ORM que proporciona Odoo, podemos crear modelos en nuestros módulos sin apenas escribir sentencias SQL y de esta forma aislar la lógica de la vista. Para ello declaramos clases que extenderán de Model, cuya clase automáticamente implementa la persistencia contra la base de datos.

Los modelos se pueden configurar con atributos, como cualquier clase, siendo `_name` el único obligatorio e importante que define el nombre del modelo. A partir de la documentación oficial de Odoo tenemos una definición mínima de un modelo.

```
from odoo import models
class MinimalModel(models.Model):
    _name = 'test.model'
```

A partir de la definición de campo podemos definir una serie de características o atributos.

Algunos atributos son:

- String, define la etiqueta en la vista.
- Required, define si el campo puede estar vacío o no.
- Help, define una ayuda para los usuarios en la vista.
- Index, odoo crea un índice para esta columna.

La anterior clase podría ser modificada de la siguiente forma con el atributo `required`.

```
from odoo import models
class MinimalModel(models.Model):
    _name = 'test.model'

    name = field.Char(required=True)
```

Una vez que ya tenemos creado un modelo, podemos añadir datos de ejemplo que además ejemplarizarán el resto de las componentes visibles.

Para ello utilizaremos el lenguaje de marcas XML y el elemento `<record>`. Cada elemento `<record>` creará o actualizará los elementos de la base de datos. Un ejemplo lo tenemos en el siguiente código:

```
<record model="{model name}" id="{record identifier}">
    <field name="{a field name}">{a value}</field>
</record>

</odoo>
```

Donde tenemos:

- Model, el nombre del modelo para la entrada.
- Id, es el identificador externo.
- Field, corresponde a los campos dentro de la definición del modelo.

El contenido de los datos de demo se actualizará en la base de datos cuando instalemos o actualicemos el módulo.

2.3.4 Acciones y menús

Las acciones y los menús nos permitirán poder navegar a través de las diferentes vistas del módulo o realizar tareas.

Las acciones pueden ser activadas de tres formas:

- Al hacer clic sobre un ítem de menú.
- Al hacer clic sobre un botón en las vistas.
- Como acciones contextuales sobre un objeto.

Un ejemplo de menú y una acción dentro del menú lo tenemos en la siguiente definición:

```
<record model="ir.actions.act_window" id="action_list_ideas">
  <field name="name">Ideas</field>
  <field name="res_model">idea.idea</field>
  <field name="view_mode">tree,form</field>
</record>
```

Como se observa, se utiliza el objeto ir.actions.act_window para incluir una nueva acción dentro de la base de datos, que en este caso abrirá un formulario.

Una vez definido, se puede utilizar esta acción dentro del menú.

El código completo sería:

```
<record model="ir.actions.act_window" id="action_list_ideas">
  <field name="name">Ideas</field>
  <field name="res_model">idea.idea</field>
  <field name="view_mode">tree,form</field>
</record>
<menuitem id="menu_ideas" parent="menu_root" name="Ideas" sequence="10"
  action="action_list_ideas"/>
```



VÍDEO DE INTERÉS

A continuación, se muestra un vídeo muy detallado de la creación de un menú personalizado:



2.3.5 Seguridad

La seguridad nos permite implementar los derechos de acceso a las diferentes partes de nuestro software y por lo tanto nos permite definir los accesos en nuestros módulos. Los derechos de acceso se definen a partir del modelo `ir.model.access`.

Cada propiedad de acceso está asociada a un modelo, grupo (puede que tenga grupo específico, y si no es así se asocia al acceso global), y por último los permisos de acceso (lectura, escritura, creación y desconexión).

Esos accesos se suelen crear en un fichero csv. Un ejemplo de ese fichero puede ser como el de la imagen:

```
id,name,model_id/id,group_id/id,perm_read,perm_write,perm_create,perm_unlink
access_idea_idea,idea.idea,model_idea_idea,base.group_user,1,1,1,0
access_idea_vote,idea.vote,model_idea_vote,base.group_user,1,1,1,0
```



EJEMPLO PRÁCTICO

Como desarrollador de software dentro de una cadena de restaurantes eres el encargado del mantenimiento y evolución del software de gestión empresarial instalado en la empresa, en este caso Odoo v13.

Tienes un módulo recién creado mediante el comando `./server/odoo-bin scaffold menus "C:\Program Files (x86)\Odoo 13.0\server\odoo\addons` y quieres crear un nuevo modelo que será un plato que tendrá un título y una descripción, así como los datos de demo y un enlace en el menú para abrir tanto el listado de platos como el formulario de creación de un nuevo plato.

Los pasos a seguir son:

1. Crear el modelo plato.

En el fichero `models/models.py` definir el modelo `Plato`:

```
from odoo import models, fields

class Plato(models.Model):
    _name = 'restaurante.plato'
    _description = "Plato de restaurante"

    name = fields.Char(string="Título", required=True)
    description = fields.Text(string="Descripción")
```

De modo que:

``_name`` --> nombre técnico de la tabla en la base de datos.

``name`` --> campo obligatorio para el título del plato.

``description`` --> campo opcional para describir el plato.

2. Crear datos de ejemplo

En el fichero `demo/demo.xml` añadir platos de prueba, estos registros se cargarán automáticamente al instalar el módulo, facilitando las pruebas.:

```
<odoo>
  <data>
    <record model="restaurante.plato" id="plato0">
      <field name="name">Paella</field>
      <field name="description">Arroz con marisco y verduras</field>
    </record>
    <record model="restaurante.plato" id="plato1">
      <field name="name">Gazpacho</field>
    </record>
  </data>
</odoo>
```

Estos registros se cargarán automáticamente al instalar el módulo, facilitando las pruebas.

3. Preparar el manifest.

En `\_\_manifest\_\_.py` añadir la referencia a las vistas y datos de seguridad, para decirle a Odoo qué ficheros debe leer para configurar menús, permisos y datos de prueba:

```
'data': [  
    'security/ir.model.access.csv',  
    'views/plato.xml',  
],  
'demo': [  
    'demo/demo.xml',  
],
```

4. Crear las vistas y menús

En `views/plato.xml` añadir la acción, para abrir la lista de platos, y añadir los menús, en la barra lateral: Restaurante --> Platos:

```
<odoo>  
  <record model="ir.actions.act_window" id="plato_list_action">  
    <field name="name">Platos</field>  
    <field name="res_model">restaurant.plato</field>  
    <field name="view_mode">tree,form</field>  
  </record>  
  <menuitem id="main_restaurante_menu" name="Restaurante"/>  
  <menuitem id="restaurant_menu" name="Restaurante"  
    parent="main_restaurante_menu"/>  
  <menuitem id="platos_menu" name="Platos"  
    parent="restaurant_menu"  
    action="plato_list_action"/>  
</odoo>
```

5. Definir permisos de acceso

En `security/ir.model.access.csv` controlar quién puede ver y modificar los platos:

id	name	model_id:id	group_id:id	perm_read	perm_write	perm_create	perm_unlink
plato_manager	plato manager	model_restaurante_plato	sales_team.group_sale_manager	1	1	1	1
plato_user	plato user	model_restaurante_plato	base.group_user	1	0	0	0

De este modo:

- Los managers pueden leer, escribir, crear y borrar.
- Los usuarios normales solo pueden consultar los platos.

6. Instalar y probar

Para terminar:

- Actualizamos la lista de aplicaciones en Odoo.
- Instalamos el módulo "Restaurante".
- Comprobamos que aparece un menú Restaurante -->Platos, con la lista de platos y un formulario para crear nuevos.

2.4 Procesamiento de datos y obtención de la información. Procesamiento de datos

Las vistas definen los mecanismos con los que un modelo en concreto es visualizado y por lo tanto sobre el que podemos realizar consultas.

Cada tipo de vista representa un tipo de visualización (un listado de campos, un gráfico, un formulario, ...). Aunque no es el ámbito de profundidad de este curso, las vistas dentro de Odoo permiten la herencia, por lo que pueden ser incluidos partes de vistas ya creados en otros módulos.

Desde Odoo 15 y 16, estas vistas también pueden desarrollarse con OWL (Odoo Web Library), lo que permite construir interfaces reactivas y más modernas.

Tal y como hemos visto anteriormente, las vistas pueden ser llamadas a través de los menús de las aplicaciones y módulos, y a través de diferentes acciones gracias a su id.

Veremos los siguientes diseños de vistas o informes básicos:

- Listados de registros.
- Formularios.
- Gráficos.

2.4.1 Herramientas para la creación de formularios e informes

De igual forma que estamos trabajando la creación de un nuevo módulo dentro de Odoo, tenemos herramientas para generar formularios, informes, gráficos y vistas en general.

Cualquier formulario, informe o vista es declarado como una nueva entrada del Modelo dentro de Odoo denominado `ir.ui.view`. Veamos un ejemplo de código extraído de la propia documentación de Odoo:

```
<record model="ir.ui.view" id="view_id">
  <field name="name">view.name</field>
  <field name="model">object_name</field>
  <field name="priority" eval="16"/>
  <field name="arch" type="xml">
    <!-- view content: <form>, <tree>, <graph>, ... -->
  </field>
</record>
```

Dentro del ejemplo tenemos las siguientes partes:

- Elemento `<record>` que permite definir un nuevo elemento en el modelo marcado por el atributo en cuestión.
- Atributo `model` permite indicar cual es el modelo donde se incluye la nueva entrada, en nuestro caso el modelo es `ir.ui.view`, que tal y como hemos dicho almacena en la base de datos las diferentes vistas.
- Elemento `<field>` permite definir los diferentes campos propios del modelo que se esté trabajando, en nuestro caso `ir.ui.view`:
 - Campo `name`, define el nombre de la vista.
 - Campo `model`, define el modelo de la vista sobre la que aplica. No confundir con el modelo `ir.ui.view`, que es el modelo que almacena los id de las vistas.
 - Campo `arch`, define el tipo de documento, en nuestro caso XML.

2.4. 2 Diseño de informes

Un informe puede definirse de múltiples formas y visualizaciones veamos algunos ejemplos de definiciones de estos informes.

- **Tree View**, en el que se muestran los campos de forma tabula. Es el informe más sencillo ya que cada registro se muestra en una fila y cada campo en una columna. Como vemos en el siguiente ejemplo, el elemento xml raíz es `<tree>`

```
<tree string="Idea list">
  <field name="name"/>
  <field name="inventor_id"/>
</tree>
```

- **Formulario (form View)**, se utilizan para crear nuevas entradas dentro de los modelos definidos. Como vemos en el siguiente ejemplo, el elemento raíz es `<form>`, e internamente se compone de una estructura de grupos, botones, campos, etc., dependiendo de la complejidad del formulario.

```
<form string="Idea form">
  <group colspan="4">
    <group colspan="2" col="2">
      <separator string="General stuff" colspan="2"/>
      <field name="name"/>
    </group>
  </group>
</form>
```

```

        <field name="inventor_id"/>
    </group>

    <group colspan="2" col="2">
        <separator string="Dates" colspan="2"/>
        <field name="active"/>
        <field name="invent_date" readonly="1"/>
    </group>

    <notebook colspan="4">
        <page string="Description">
            <field name="description" nolabel="1"/>
        </page>
    </notebook>

    <field name="state"/>
</group>
</form>

```

- **Gráficos (Graph View)**, se utilizan para diseñar agregados, análisis y representaciones gráficas de los datos dentro de un modelo. Como vemos en el siguiente ejemplo, el elemento raíz es <graph> y permite definir tipos de visualizaciones dependiendo del atributo type (barras, línea, tarta, ...)

```

<graph string="Total idea score by Inventor">
    <field name="inventor_id"/>
    <field name="score" type="measure"/>
</graph>

```



EJEMPLO PRÁCTICO

Como desarrollador de la cadena de restaurantes, necesitas que el personal pueda imprimir una “Carta del día” con la lista de platos seleccionados para cada Carta, mostrando también el total de calorías.

Usarás un informe QWeb enlazado al modelo `restaurant.cartas` (ya creado en los ejemplos previos) y lo añadirás al menú para que sea accesible desde la interfaz.

Los pasos a seguir son:

1. Añadir la relación entre Carta y Plato

Si aún no existe relación, crea en `models/models.py` un campo para asociar varios platos a una carta:

```

from odoo import models, fields
class Carta(models.Model):
    _name = 'restaurant.cartas'
    _description = 'Menú de restaurante'

```



```
name = fields.Char(string="Título", required=True)
description = fields.Text()
start_date = fields.Date(string="Fecha de inicio")
duration = fields.Float(digits=(6, 2), help="Activo en días")
calorias = fields.Integer(string="Calorías (referencia)")
responsible_id = fields.Many2one('res.users', ondelete='set
null',
                                string="Responsable",
                                index=True)

platos_ids = fields.Many2many(
    'restaurant.plato',
    string='Platos incluidos'
)
```

Se usará Many2many porque una carta puede incluir muchos platos y un mismo plato puede aparecer en distintas cartas.

2. Declarar la acción de informe

En `views/restaurant.xml` (o un `reports/restaurant\_report.xml` si prefieres separarlo), añade la definición del informe y su plantilla QWeb:

```
<odoo>
<!-- Acción del informe -->
<report
    id="report_carta_dia"
    model="restaurant.carta"
    string="Imprimir Carta del día"
    report_type="qweb-pdf"
    name="restaurant.report_carta_dia"
    file="restaurant.report_carta_dia"
    print_report_name="'Carta_' + object.name"/>

<!-- Plantilla QWeb -->
<template id="report_carta_dia">
    <t t-call="web.external_layout">
        <t t-set="doc" t-value="doc"/>
        <div class="page">
            <h2>Carta del día: <t t-esc="doc.name"/></h2>
            <p>Fecha de inicio: <t t-esc="doc.start_date or ''"/></p>
            <p>Responsable: <t t-esc="doc.responsible_id.name or '-
''"/></p>

            <h3>Platos</h3>
            <table class="table table-sm o_main_table">
                <thead>
                    <tr>
                        <th>Plato</th>
                        <th>Descripción</th>
                        <th class="text-right">Calorías (ref.)</th>
                    </tr>
                </thead>
                <tbody>
                    <t t-foreach="doc.platos_ids" t-as="pl">
                        <tr>
                            <td><t t-esc="pl.name"/></td>
                            <td><t t-esc="pl.description or ''"/></td>
                            <td class="text-right">
```

```

        <!-- Si guardas calorías por plato, muéstralas
aquí.
        Si no, dejamos vacío o 0 como referencia. -->
        <t t-esc="getattr(pl, 'calorias', 0)"/>
    </td>
</tr>
</t>
</tbody>
</table>

    <t t-set="total_cal"
        t-value="sum([getattr(pl, 'calorias', 0) for pl in
doc.platos_ids])"/>
    <p class="text-right">
        <strong>Total calorías:</strong>
        <t t-esc="total_cal"/>
    </p>
</div>
</t>
</template>
</odoo>

```

Explicación del código:

- `<report ...>` registra el informe y lo asocia al modelo `restaurante.carta`.
- La plantilla QWeb monta el HTML del PDF. Usamos expresiones `t-esc` y `t-foreach` para pintar datos.
- Calculamos un total con Python embebido (`sum(...)`) sobre los platos.
- Si todavía no guardas calorías por plato, puedes dejar el total a 0 o añadir el campo `calorias` también a `restaurante.plato`.

3. Añadir el informe al menú de Carta

Añade un botón Imprimir en el formulario de Carta o un menú contextual, que llama a la acción del informe usando el external id `%(report\_carta\_dia)d`, consiguiendo que el PDF sea accesible desde el propio formulario, muy usable para el personal de sala.:

```

<odoo>
<!-- Botón en el formulario de Carta -->
<record id="carta_form_view" model="ir.ui.view">
    <field name="name">carta.form</field>
    <field name="model">restaurante.carta</field>
    <field name="arch" type="xml">
        <form string="Menú">
            <sheet>
                <group>
                    <field name="name"/>
                    <field name="start_date"/>
                    <field name="duration"/>
                    <field name="calorias"/>
                    <field name="responsible_id"/>
                    <field name="platos_ids" widget="many2many_tags"/>
                </group>
            </sheet>
            <header>
                <button name="%(report_carta_dia)d" string="Imprimir Carta
del día"
                    type="action" class="btn-primary"/>
            </header>
        </form>
    </field>
</record>

```

```
</field>
</record>
</odoo>
```

4. Manifiesto y seguridad

Asegúrate de listar los ficheros en el `\_\_manifest\_\_.py` y de tener permisos adecuados:

```
'data': [
    'security/ir.model.access.csv',
    'views/restaurante.xml', # o 'reports/restaurante_report.xml'
],
```

Y en `security/ir.model.access.csv` incluye acceso de lectura para usuarios base y total para managers (como ya hacías con Carta y Plato).

5. Probar

1. Actualizar lista de aplicaciones e instalar/actualizar el módulo.
2. Ir a Restaurante --> Menús (Cartas), abrir una carta, añadir platos y pulsar "Imprimir Carta del día".
3. Verifica que el PDF muestra la lista y el total de calorías.

2.5 Operaciones de consulta. Herramientas

Hemos visto como dentro de un módulo de Odoo implementamos los diferentes modelos que nos permiten guardar y extraer información de forma persistente desde la base de datos.

Un modelo puede necesitar datos de otra aplicación, módulo o de otro modelo. Por ejemplo, los pedidos de venta están relacionados con los datos del cliente.

Para poder relacionar datos de dos módulos tenemos las siguientes relaciones:

- **Many2one:** en este caso tenemos la relación entre dos modelos de 1 a n elementos. La sintaxis extraída de la documentación de fabricante:

```
many2one(other_model, ondelete='set null')
```

- **One2many:** en este caso tenemos la relación entre dos modelos de n a 1 elementos. En este caso para que se pueda tener una relación one2many en un modelo, necesitamos que exista una relación many2one en el primer modelo. La sintaxis extraída de la documentación de fabricante:

```
one2many(other_model, related_field)
```

- **Many2many:** en este caso tenemos la relación bidireccional entre dos modelos de n a n elementos.



EJEMPLO PRÁCTICO

Como desarrollador de software dentro de una cadena de restaurantes eres el encargado del mantenimiento y evolución del software de gestión empresarial instalado en la empresa, en este caso Odoo v13.

Tienes un modulo recién creado mediante el comando `./server/odoo-bin scaffold menus "C:\Program Files (x86)\Odoo 13.0\server\odoo\addons"`, en el que se ha desarrollado un modelo de platos. Necesitamos poder crear nuevos menús con una fecha de inicio y una duración del menú, un responsable del menú que será un usuario y además añadir al enlace tanto el listado como el nuevo formulario.

Los pasos a seguir son:

1. Crearemos un nuevo modelo para tener el Menu que comienza a estar activo en una fecha, por un período de tiempo y con unas calorías. El fichero `models.py` queda de la siguiente forma:

```
# -*- coding: utf-8 -*-

from odoo import models, fields, api

class Plato(models.Model):
    _name = "restaurante.plato"
    _description = "Plato de restaurante"

    name = fields.Char(string="Title", required=True)
    description = fields.Text()

class Carta(models.Model):
    _name = 'restaurante.carta'
    _description = "Menu de restaurante"

    name = fields.Char(string="Title", required=True)
    description = fields.Text()
    start_date = fields.Date()
    duration = fields.Float(digits=(6, 2), help="Activo en días")
    calorías = fields.Integer(string="Calorías")
```

2. Añadiremos el responsable de la carta usando `many2one` dentro del modelo carta:

```
responsable_id = fields.Many2one('res.users',
    ondelete='set null', string="Responsable", index=True)
```

- `Carta` es un nuevo modelo independiente de `Plato`.
- `Responsable\_id` usa `Many2one` --> relación con la tabla de usuarios (`res.users`), es decir, cada menú tendrá un responsable asignado.

- Renombraremos plato.xml a restaurante.xml para una mejor comprensión y añadiremos en primer lugar la vista, en `views/restaurante.xml`, para que aparezca una nueva opción Restaurante --> Menús, desde donde podremos ver listados y formularios:

```
<record model="ir.actions.act_window" id="menu_list_action">
  <field name="name">MEnus</field>
  <field name="res_model">restaurante.carta</field>
  <field name="view_mode">tree,form</field>
</record>

<menuitem id="carta_menu" name="Menus"
  parent="restaurante_menu"
  action="menu_list_action" sequence="2"
/>
```

- Y en segundo lugar un listado personalizado, creando una vista de formulario para 'Carta', que permitirá introducir los datos de cada menú como son nombre, fecha de inicio, duración, calorías y responsable:

```
<!-- session form view -->
<record model="ir.ui.view" id="carta_form_view">
  <field name="name">carta.form</field>
  <field name="model">restaurante.carta</field>
  <field name="arch" type="xml">
    <form string="Menu Form">
      <sheet>
        <group>
          <field name="name"/>
          <field name="start_date"/>
          <field name="duration"/>
          <field name="calorias"/>
          <field name="responsible_id"/>
        </group>
      </sheet>
    </form>
  </field>
</record>
```

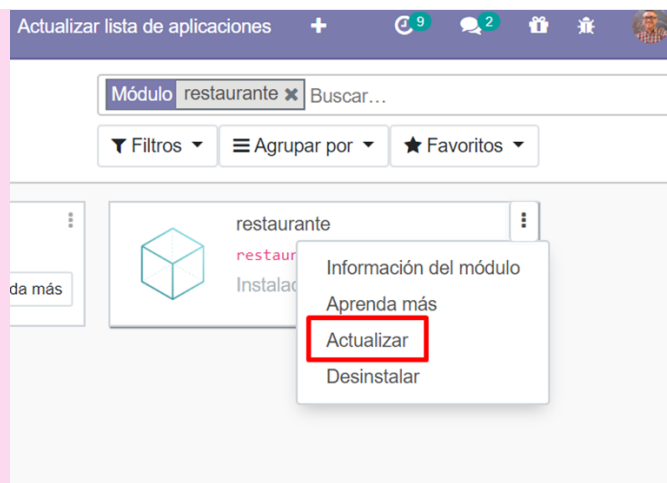
- Añadimos las configuraciones de seguridad, como son los permisos, en En `security/ir.model.access.csv`

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
session_manager,session
manager,model_restaurante_plato,sales_team.group_sale_manager,1,1,1,1
session_manager,session manager,model_restaurante_carta,sales_team.group_sale_manager,1,1,1,1
session_read_all,session all,model_restaurante_carta,1,0,0,0
```

De este modo:

- Los managers pueden crear, modificar y borrar.
- Los usuarios normales solo pueden consultar los menús.

- Instalar y probar. Actualizar el módulo:



Actualización del módulo.

Fuente: elaboración propia.

1. Actualizar la lista de aplicaciones
2. Instalar el módulo
3. Comprobar que aparece el menú Restaurante-->Menús con las opciones de listado y formulario.

2.5.1 Técnicas de optimización de consultas y acceso a grandes volúmenes de información

Cuando se realizan consultas a grandes bases de datos, se nos presenta la pregunta de cómo poder optimizar las consultas para que el rendimiento del sistema mejore o sea el óptimo.

Para comenzar debemos seguir dos grandes consejos siempre:

- En el caso de que seamos propietarios y administradores de la base de datos, es nuestro caso con Odoo, se debe intentar realizar procedimientos almacenados en la base de datos. Un procedimiento almacenado consiste en una consulta o conjunto de funciones que se ejecutan en la base de datos y que al estar dentro de la misma base de datos siempre será mucho más rápida que ser lanzada desde una aplicación externa.
- Generar vistas dentro de la base de datos PostgreSQL. Es otro mecanismo dentro de la propia base de datos que nos permite agrupar la información para poderla alcanzar de una forma rápida, ya que normalmente esa información procede de varias tablas.
- Por último, intentar realizar el mínimo número de consultas posibles desde la aplicación. Aquí también existen muchas técnicas, que han evolucionado en la

actualidad y que permiten mantener una serie de información en el lado del cliente sin necesidad de ser actualizada.

Evidentemente, la combinación de varias de las técnicas descritas anteriormente es posible y necesaria.

Normalmente se siguen las siguientes premisas:

- Realizar procedimientos almacenados y vistas en el caso de consultas que se conoce sus parámetros de entrada y que por lo tanto no dependen de los datos que proporcione el usuario para formar la información.
- Almacenar en cache los datos que son usados en la sesión o datos cuyo valor no cambia durante la sesión de trabajo.
- Recoger el máximo posible de información en una consulta para ser tratada posteriormente al estar ya almacenada en memoria.

Como podemos observar, la máxima, es intentar no realizar muchas consultas a la base de datos, ya que este medio, es el más lento e ineficiente dentro de la cadena.



ARTÍCULO DE INTERÉS

En el siguiente artículo dispondremos de más información sobre los procedimientos almacenados:



2.5.2 Generación de programas de extracción de datos entre sistemas (batch inputs)

La extracción de datos entre sistemas, pasarelas o adaptadores entre software de gestión empresarial son procesos importantes y comunes dentro de las empresas.

En una empresa nos podemos encontrar con diferentes software de gestión de la información, bien porque se han realizado compras e implantaciones de software específico para las necesidades de departamentos, bien porque el software ha ido evolucionando. La extracción e importación (batch inputs) suelen ser procesos muy habituales programados para realizar alguna de las siguientes tareas:

- Extracción automática de información para la generación de informes, suele producirse cuando se usan sistemas de BI, o generadores de cuadros de mandos, donde se suele utilizar bases de datos intermedias que preparan la información para poder después realizar los informes.
- Extracción automática de la información para el uso en un software externo. Es un caso muy parecido al anterior, solo que en este caso los datos se suelen utilizar en un segundo software de gestión empresarial que generará nueva información a partir de los datos del primer software.
- Importación automática o batch input, en este caso nuestro software de gestión empresarial necesita de unos datos que provienen de software externos para que los usuarios puedan realizar sus tareas.

La programación de estos programas pasarela, suele ser a través de scripts externos que trabajan directamente con la base de datos y que tienen las siguientes características:

- Realizan la extracción de la información de la base de datos mediante secuencias SELECT.
- Realizan la importación de la información en otra base de datos mediante secuencias UPDATE e INSERT.
- Suelen ser scripts programados y automáticos en momentos temporales donde no haya trabajo o carga con los sistemas.



RECUERDA

En la UD4 vimos cómo se realizaba el proceso de extracción de información a través de JasperStudio, generando informes particularizados.



VÍDEO DE INTERÉS

En el siguiente vídeo podrás ver un buen ejemplo sobre automatización de procesos:



2.6 Depuración y tratamiento de errores

Depurar un programa consiste en encontrar y corregir los problemas que pueda tener el código del mismo.

Podemos hacerlo por métodos “arcaicos” usando una herramienta básica como es la función `print()` para ir viendo los valores de las variables relevantes del programa, o ir imprimiendo texto por pantalla cada cierto número de instrucciones (sobre todo cuando las instrucciones llevan a una bifurcación como un `if else`) para saber por qué ramas del código transcurre la ejecución y qué valores van tomando las diferentes variables.

También contamos con herramientas más poderosas como es el Depurador, que nos ayuda a detectar y corregir los fallos más rápidamente al permitir la ejecución interactiva del programa.

Como inconveniente del Depurador, hay que indicar que ralentiza la ejecución del código haciendo que un simple programa pueda durar varios minutos, pero hay técnicas para minimizar este punto. Utilizarlo nos proporciona una información muy útil que compensa con creces este inconveniente, hay errores que son muy difíciles de detectar y corregir con el uso exclusivamente de la función `print()`.

Odoo es un software que se ejecuta en modo cliente/servidor y podemos usar diferentes mecanismos para poder realizar un debugging de la aplicación:

- Con ``print()`` o ``logging`` en Python (logs controlados desde ``odoo.conf``).
- Usando el depurador ``pdb`` o la integración de depuración de Visual Studio Code.

- Activando el modo desarrollador desde la propia interfaz de Odoo o añadiendo `?debug=1` en la URL.
- De forma avanzada, usando el Odoo Debug Manager (extensión para navegadores) o las herramientas de desarrollador del propio navegador.

Este último mecanismo es muy interesante ya que nos permite añadir un recubrimiento sin realizar cambios en el código fuente. Activaremos el modo desarrollador desde Ajustes --> Activar modo desarrollador en Odoo, o instalaremos extensiones de navegador como Odoo Debug Manager, disponibles para Chrome y Firefox.

Para ello seguiremos los siguientes pasos:

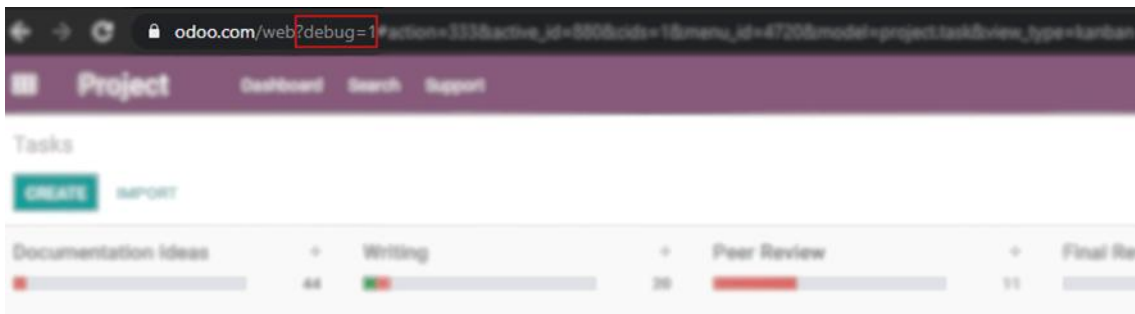
1. Instalaremos el plugin de Chrome, Odoo Debug.



Activación del Odoo Debug en Chrome.

Fuente: https://www.odoo.com/documentation/13.0/es/_images/monkey.png

2. Podremos usar el debug añadiendo a la url de cada una de las páginas en modo desarrollador ?debug=1 o ?debug=true



Activación a través de URL.

Fuente: https://www.odoo.com/documentation/13.0/es/_images/url.png



ENLACE DE INTERÉS

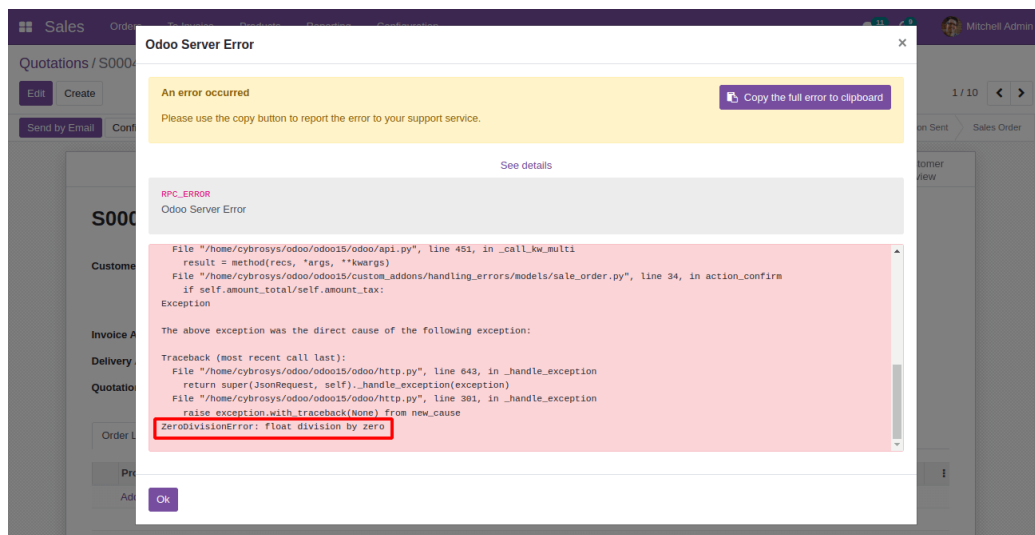
A través del siguiente enlace podrás instalar el plugin de Debug sobre Odoo:



2.6.1 Manejo de errores

Otra de las utilidades que tenemos con odoo en modo desarrollador es la de poder recibir errores cuando estos se producen durante el desarrollo. Errores que suelen ser muy comunes.

Cuando desarrollamos un módulo, durante su instalación o durante su actualización pueden aparecer ventanas de error como las que vemos en la imagen:



Ventana de error.

Fuente: <https://www.images.cybrosys.com/blog/Uploads/BlogImage/error-handling-in-odoo-15-2.png>

Cuando abrimos los detalles de esta ventana solemos encontrar 3 partes bien diferenciadas:

- Una primera parte extensa y detallada de todo el itinerario de los errores producidos dentro de la aplicación.

- Un mensaje específico del error concreto y donde se produce.
- Un contexto específico del error.

A partir de esta información debemos corregir el error, y en este punto no existe una metodología única. El primer parámetro importante es la experiencia del desarrollador y la experiencia con el software lo que permitirá corregir los errores.

Un pequeño proceso y metodología puede ser el siguiente:

1. Abrir el contexto que indica el mensaje del error, suele ser un fichero dentro del módulo.
2. Realizar cambios pequeños y comprobar si se ha resuelto el problema.
3. Si no se resuelve el problema de una forma corta y sencilla, realizar una vuelta atrás del código y comprobar el correcto funcionamiento.



EJEMPLO PRÁCTICO

Tras crear el modelo ``restaurantecarta``, al abrir el formulario aparece un error diciendo que un campo no existe.

Te han solicitado que busques dónde está la referencia incorrecta y la modifiques para que funcione de forma correcta.

Los pasos a seguir son:

1. Reproducir el error:

Abre Restaurante --> Menús (Cartas) y entra en una carta.

Salta un mensaje tipo:

``Field 'calories' does not exist in model 'restaurantecarta'`` (o similar, por un typo o un campo borrado).

Este suele ser el fallo más común al editar vistas: el nombre del campo en la vista no coincide con el del modelo.

2. Activar el modo desarrollador:

- Activa el modo desarrollador (desde Ajustes o añadiendo ``?debug=1`` en la URL).
- Vuelve a abrir el formulario para que aparezca el traceback.

- El objetivo es ver el ID de la vista que está fallando y el nombre exacto del campo que no encuentra.

3. Localizar la línea problemática:

En el mensaje de error verás algo como:

```
`view: restaurante.carta (id=carta_form_view) ... <field name="calories"/>`
```

Con eso ya sabes que la vista `carta\_form\_view` tiene un ``<field name="calories"/>`` pero el modelo define `calorias` (en español) o simplemente no tiene ese campo.

4. Corregir:

Tienes dos opciones posibles de corrección:

Opción A: corregir la vista (rápido y didáctico)

Edita `views/restaurante.xml` y cambia el nombre del campo para que coincida con el modelo:

```
<field name="calorias"/> <!-- en vez de 'calories' -->
```

Opción B: crear el campo en el modelo (si realmente lo necesitas)

Edita `models/models.py`

```
from odoo import models, fields

class Carta(models.Model):
    _name = 'restaurante.carta'
    _description = "Menú de restaurante"

    # ...
    calorias = fields.Integer(string="Calorías") # asegurar que existe
```

5. Actualizar el módulo y comprobar

- Actualizar lista de aplicaciones e instalar/actualizar el módulo.
- Abre de nuevo la carta: el formulario debería cargarse sin errores.

RESUMEN FINAL

En esta unidad hemos tratado en tema del desarrollo de componentes y módulos para los sistemas ERP-CRM, entendiendo que cada empresa presenta necesidades particulares que a menudo requieren adaptaciones sobre la base del software estándar, comenzando con el análisis de la arquitectura de los sistemas ERP-CRM, diferenciando entre software estándar, híbrido y a medida, así como las metodologías más empleadas en la actualidad, como las ágiles, que permiten una planificación flexible y cercana a la realidad de la empresa.

A continuación, hemos visto el desarrollo de módulos, conociendo los entornos y herramientas necesarios para trabajar en este ámbito y repasando los lenguajes más habituales, entre los que destaca Python para la lógica de negocio, junto con otros utilizados en la construcción de vistas e interfaces. A partir de aquí, estudiamos la creación de formularios e informes, entendidos como piezas clave para que los usuarios interactúen con el sistema y accedan a la información que necesitan en su día a día.

Mención aparte a las extracciones de información y operaciones de consulta, aspectos fundamentales para el análisis de datos y la toma de decisiones, donde cobra importancia la optimización de las consultas y el uso de técnicas que permitan gestionar grandes volúmenes de información de manera eficiente.

Para finalizar con la depuración de programas y el tratamiento de errores, resaltando la importancia como tarea esencial en cualquier proceso de desarrollo, de cara a asegurar la fiabilidad y estabilidad de los sistemas. Haciendo un recorrido por los métodos básicos y avanzados de depuración, así como la forma de identificar y corregir fallos para mejorar el funcionamiento de los módulos creados.